



哈爾濱工業大學  
HARBIN INSTITUTE OF TECHNOLOGY



# 分布式系统核心 (II)

## Raft共识算法与领导者选举

王宏志

哈尔滨工业大学

计算学部 海量数据计算研究中心

wangzh@hit.edu.cn

<http://homepage.hit.edu.cn/wang>

# 目录

## Contents

- 1 Raft共识算法
- 2 领导者选举
- 3 日志复制
- 3 安全性

# 目录

## Contents

- 1 Raft共识算法
- 2 领导者选举
- 3 日志复制
- 3 安全性

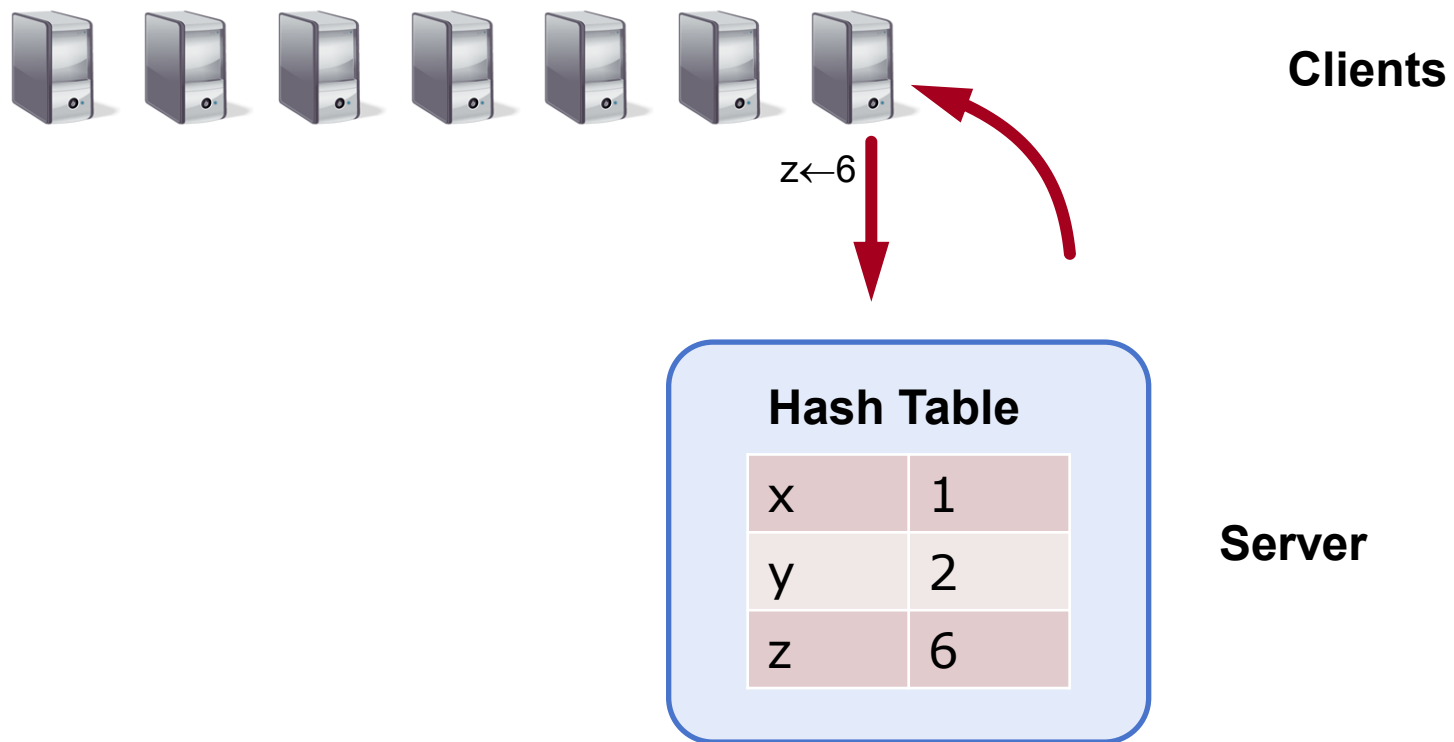
# 什么是共识?

- **共识:让多个服务器就状态达成一致**
- **解决方案通常处理少数服务器故障**
- **主从复制,可以安全自主地从主故障中恢复**
- **用于构建一致的存储系统**
  - 顶级系统配置
  - 有时管理整个数据库状态(例如,Spanner)
- **例如:Chubby、ZooKeeper、Doozer**

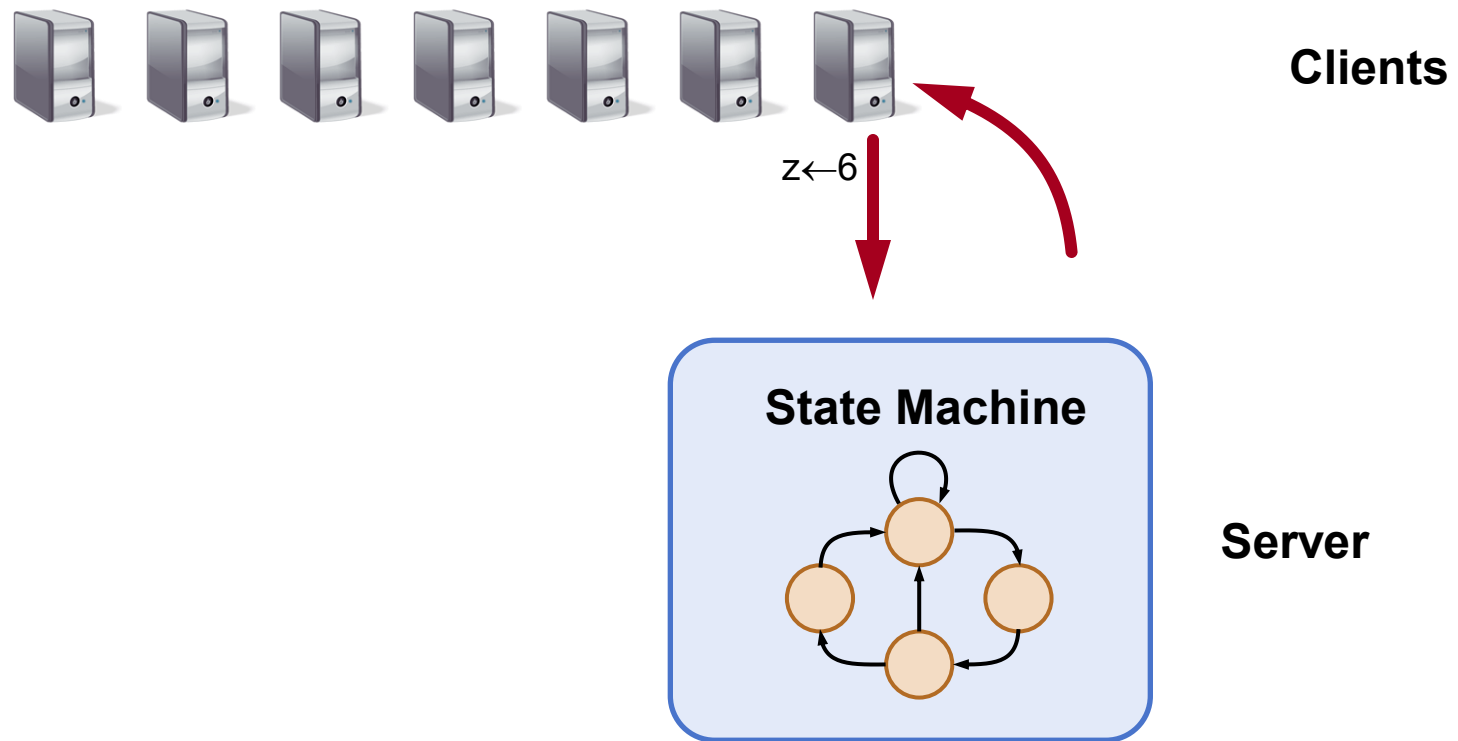
# Raft:使共识更容易

- **共识被广泛认为是困难的**
  - 由一种名为 Paxos 的算法主导
- **Raft旨在使之易于理解**
  - 用户研究表明学生学习 Raft 更好
- **在 GitHub 上进行25+ Raft 的实现**
  - 见 <http://raftconsensus.github.io>
  - Bloom、C#、C++、Clojure、Elixir、Erlang、F#、Go、Haskell、Java、Javascript、OCaml、Python、Ruby

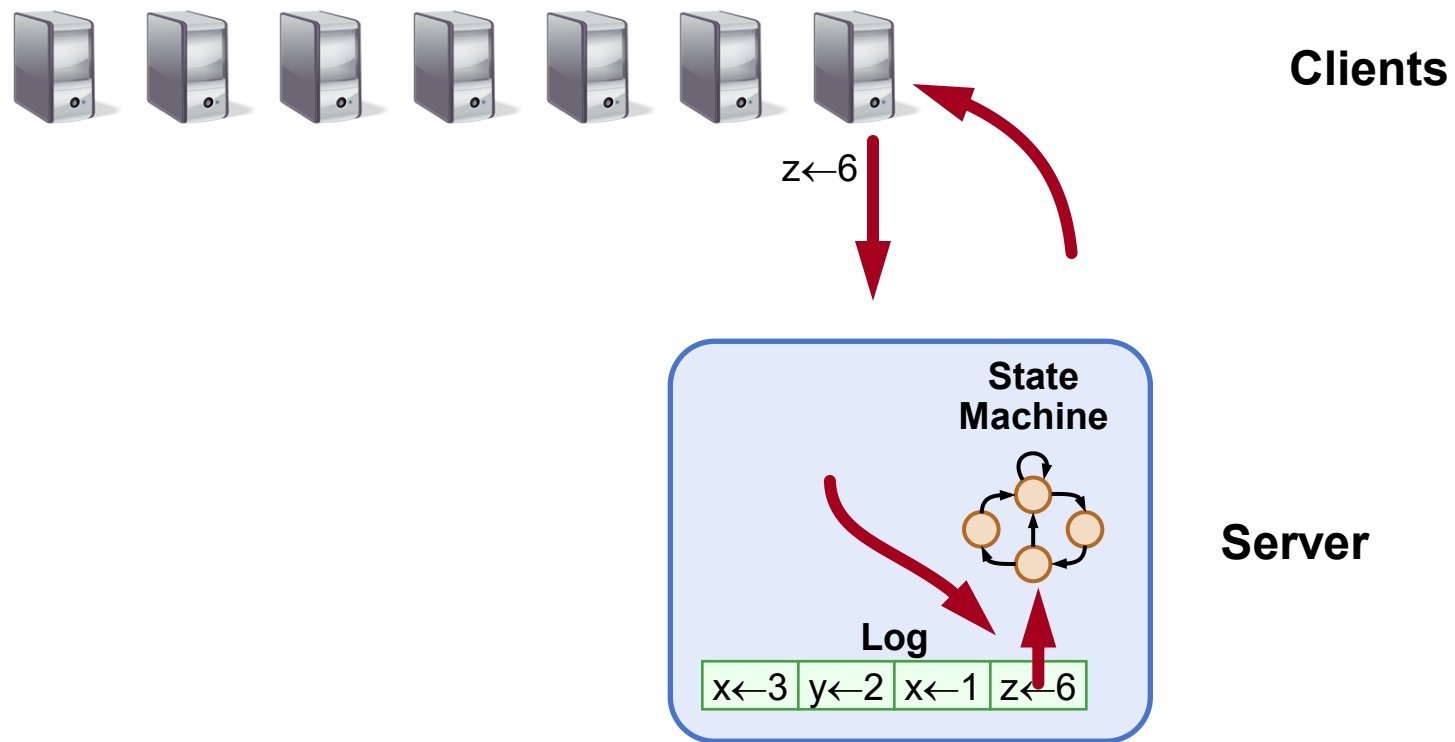
# 单服务器



# 单服务器

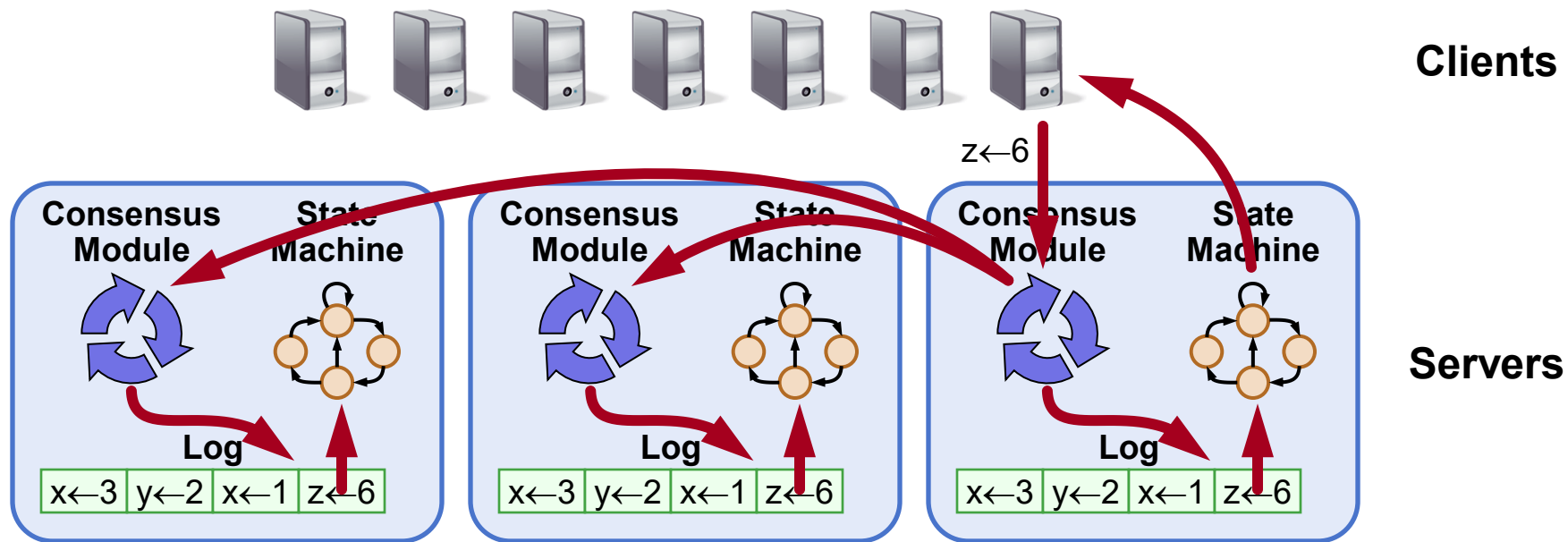


# 单服务器





## 目标:复制日志



- **复制日志⇒复制状态机**
  - 所有服务器都以相同的顺序执行相同的命令
- **共识模块确保正确的日志复制**
- **只要大多数服务器都启动,系统就会进步**
- **故障模型:故障停止(非拜占庭式)、延迟/丢失消息**

# 达成共识的方法

## 达成共识的两种大方法:

- **对称,无首领:**

- 所有服务器都有平等的角色
- 客户端可以联系任何服务器

- **不对称,以领导者为本:**

- 在任何时候,一个服务器负责,其他服务器接受它的决定
- 客户与领导者沟通

- **Raft 使用领导者:**

- 分解问题(正常运行,领导者更改)
- 简化正常操作(无冲突)
- 比无领导者的方法更有效

## 1. 领袖选举:

- 选择其中一个服务器作为领导者(Leader)
- 检测碰撞事故,选择新的领导者

## 2. 正常操作(日志复制)

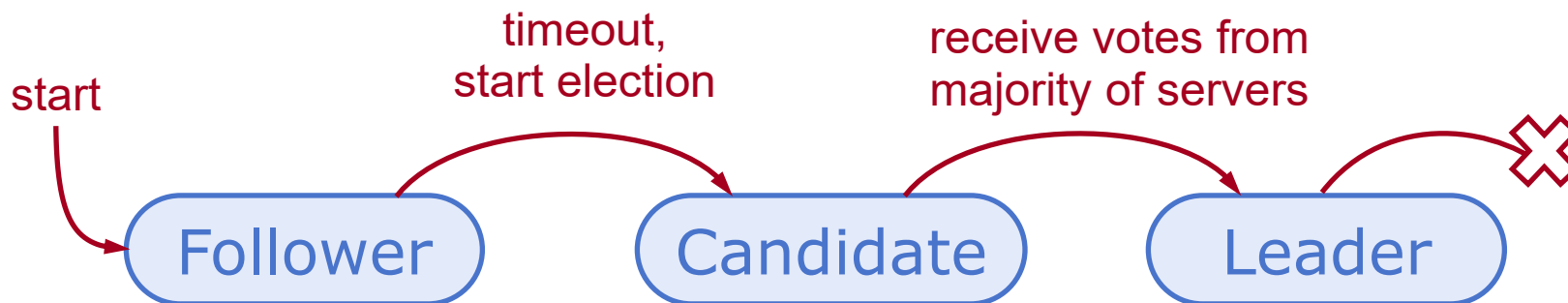
- Leader 从客户端获取命令,将其附加到日志中
- Leader 将其日志复制到其他服务器(覆盖不一致)

## 3. 安全性保障

- 需要承诺的条目才能在领导者更替中生存
- 定义承诺规则,操纵领导者选举

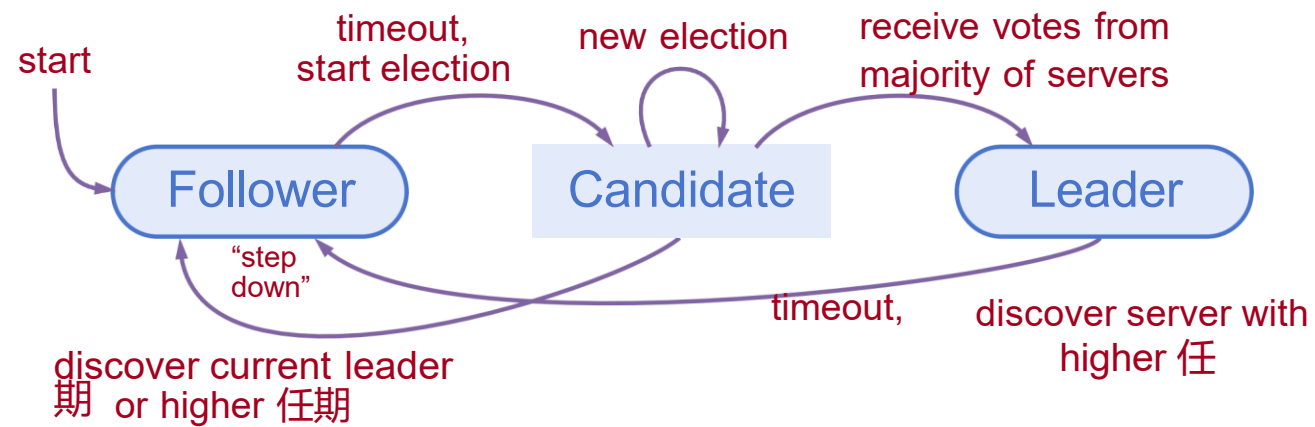
# 服务器状态

- 在任何时候,每个服务器都是三者之一:
  - **Leader**:处理所有客户端交互,日志复制
  - 一次最多一个可行的领导者
  - **Follower**:完全被动的副本(不发出 RPC,只响应传入的 RPC)
  - **Candidate**:用来选举新领袖

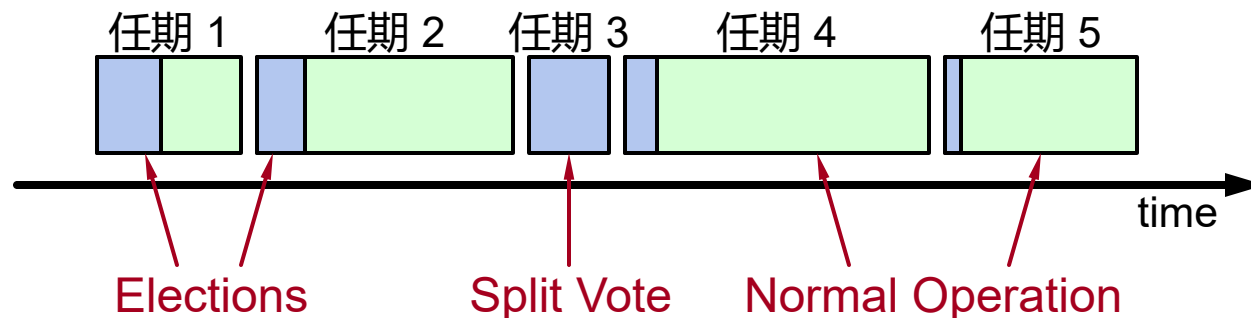


- **服务器以关注者身份启动**
- **追随者期望从领导者或候选人那里获得 RPC**
- **如果选举超时已过而没有 RPC:**
  - 追随者假设领导者已崩溃
  - 追随者启动新选举
  - 超时通常为100-500毫秒
- **领导者必须发送心跳以维持权威**

# 活动验证



# 任期



- **时间分为任期：**
  - 选举
  - 在一个领导者下正常运作
- **每任最多一人**
- **有些任期没有领袖(选举失败)**
- **每个服务器都保持当前期限值**
- **任期关键作用:识别过时的信息**

# 目录

## Contents

- 1 Raft共识算法
- 2 领导者选举**
- 3 日志复制
- 3 安全性



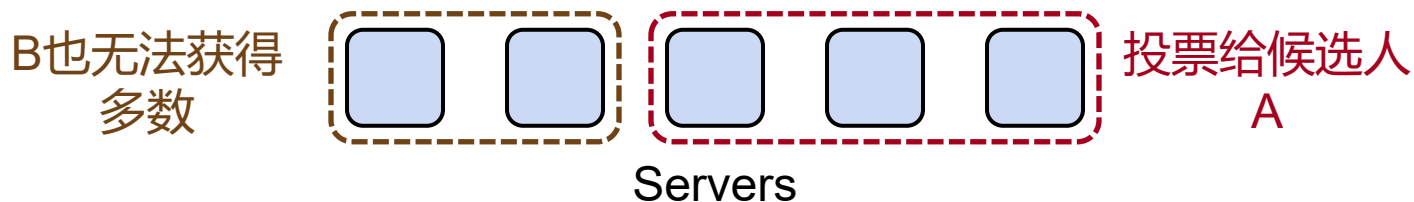
## 选举暂停时:

- 递增当前任期号
- 改为候选人
- 投票给自己
- 将 **RequestVote RPC** 发送到所有其他服务器,等待:
  1. 获得超过半数服务器的投票:
    - 成为领导者
    - 将 AppendEntries 心跳发送到所有其他服务器
  2. 从有效领导者那里获得 RPC:
    - 返回从者状态
  3. 无人胜出(选举暂停已过):
    - 递增任期,开始新选举

## 选举属性

- **安全:每任期最多允许一名获胜者**

- 每个服务器每期只给一票(在磁盘上保留)
- 两个不同的候选人不能在任一任期内积累多数



- **活跃度:某个候选人最终必须获胜**

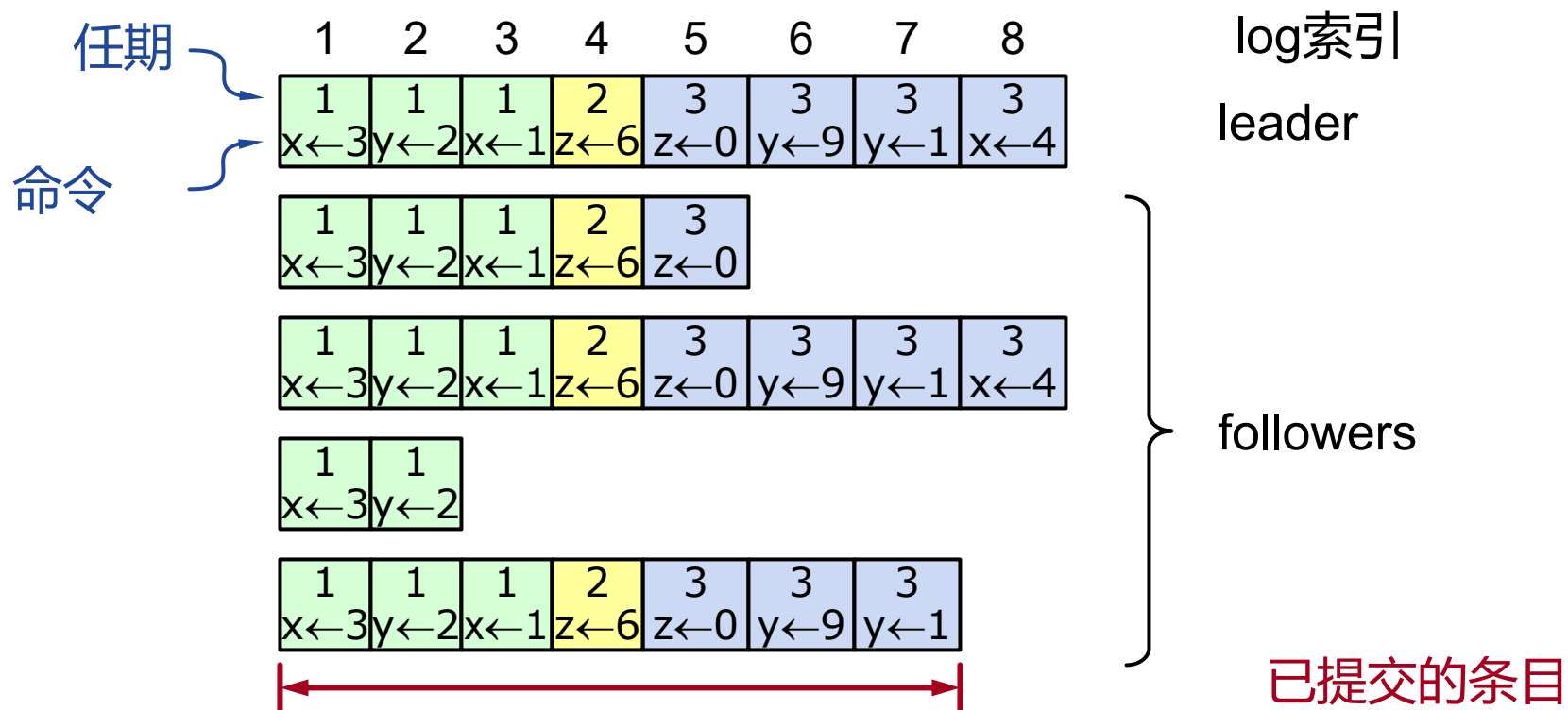
- 从100-200ms范围内随机选择选举超时
- 通常,一个服务器超时,在其他服务器醒来之前赢得选举

# 目录

## Contents

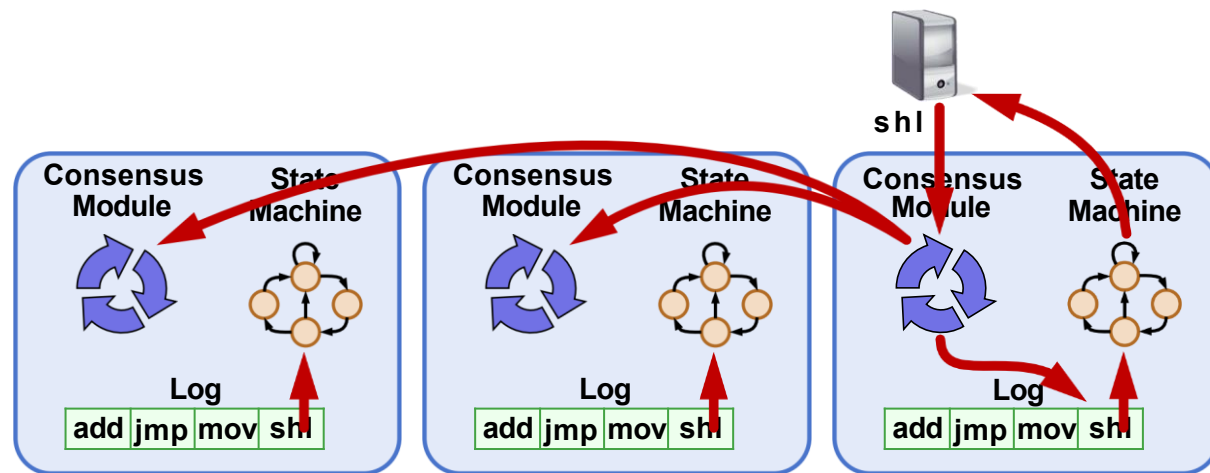
- 1 Raft共识算法
- 2 领导者选举
- 3 日志复制
- 3 安全性

# 日志结构



- 日志条目=索引, 任期, 命令
- 日志存储在稳定存储(磁盘)上; 在崩溃中幸存下来

- 客户向领导者发出命令
- 领导者将命令附加到其日志中
- Leader 将 AppendEntries RPC 发送给追随者
- 一旦新条目安全提交：
  - Leader 将命令应用于其状态机,将结果返回给客户端
- 在后台追赶追随者：
  - Leader 在后续的 AppendEntries RPC 中通知已提交的条目
  - 从者将已提交的命令应用于其状态机



- 崩溃 /耗时的followers?

- 领导者重试 RPC,直到成功为止

- 性能在常见情况下是最佳的:

- 一个成功的 RPC 到任何大多数服务器

## 日志之间的高度一致性:

- 如果不同服务器上的日志条目具有相同的索引和术语:

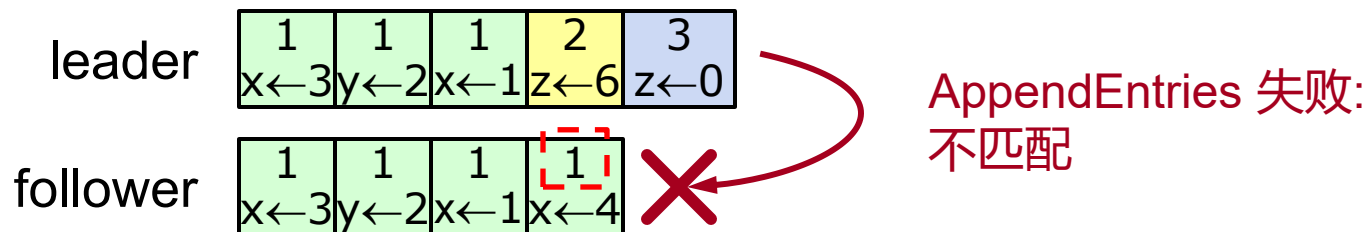
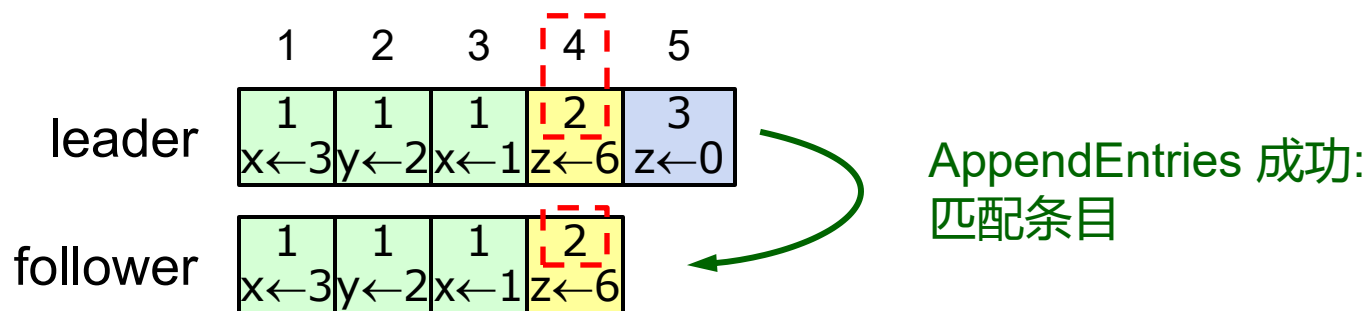
- 他们存储相同的命令
- 日志在前面的所有条目中都是一样的

| 1        | 2        | 3        | 4        | 5        | 6        |
|----------|----------|----------|----------|----------|----------|
| 1<br>x←3 | 1<br>y←2 | 1<br>x←1 | 2<br>z←6 | 3<br>z←0 | 3<br>y←9 |
| 1<br>x←3 | 1<br>y←2 | 1<br>x←1 | 2<br>z←6 | 3<br>z←0 | 4<br>x←4 |

- 如果提交了给定的条目,则前面的所有条目也都被提交了

## AppendEntries 一致性检查

- 每个 AppendEntries RPC 都包含索引,在新条目之前的条目
- Follower 必须包含匹配的条目; 否则,它拒绝请求
- 实施归纳步骤,确保连贯性





## 领导者变更

- 新领导日志是真值，没有特殊步骤，开始正常操作
- 最终将使从者的日志与领导者的日志相同
- 旧领导可能留下了部分复制的条目

| log index      | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|----------------|---|---|---|---|---|---|---|
| term           |   |   |   |   |   |   |   |
| S <sub>1</sub> | 1 | 1 | 5 | 6 | 6 | 6 |   |
| S <sub>2</sub> | 1 | 1 | 5 | 6 | 7 | 7 | 7 |
| S <sub>3</sub> | 1 | 1 | 5 | 5 |   |   |   |
| S <sub>4</sub> | 1 | 1 | 2 | 4 |   |   |   |
| S <sub>5</sub> | 1 | 1 | 2 | 2 | 3 | 3 | 3 |

一旦日志条目应用于状态机,其他状态机就不必为该日志条目应用不同的值

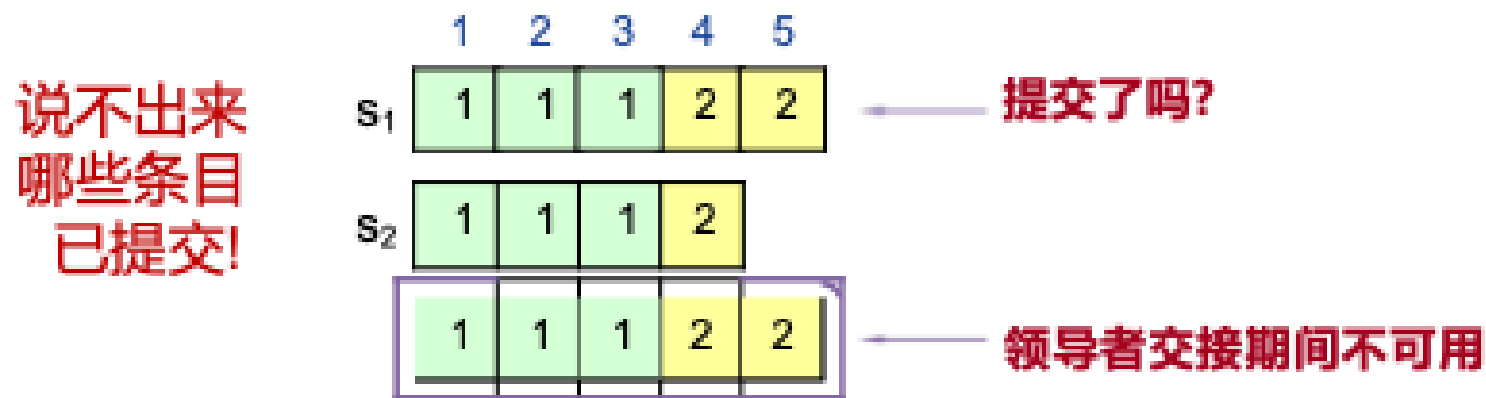
- Raft安全属性:如果领导者已决定日志条目已提交, 则该条目将出现在所有未来领导者的日志中。
- 为什么这能保证更高层次的目标?
  1. 领导者永远不会覆盖日志中的条目
  2. 只有领导者日志中的条目可以被提交
  3. 条目必须在应用到状态机之前被提交

**Committed → Present in future leaders' logs**

Restrictions on  
commitment

Restrictions on  
leader election

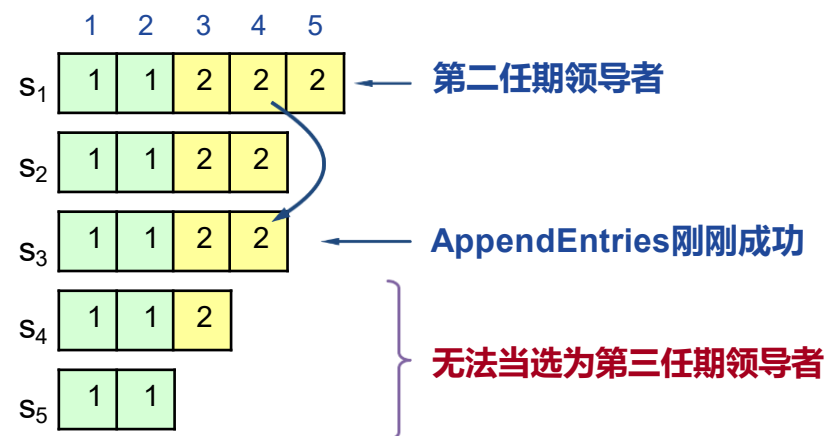
## 选择最佳领导者



选举最有可能包含所有已提交日志条目的候选者

- 在 RequestVote RPC 中，候选者会携带其最后一条日志的索引 (index) 和任期 (任期) ；
- 如果投票者 V 发现自己的日志 “更完整”，则会拒绝投票，判定规则为：
  - 投票者的最后日志条目具有更新的任期 (任期 更大)，或
  - 在任期相同的情况下，投票者的日志条目索引更大 (index 更高) ；
- 因此，当选的 Leader 一定在参与选举的多数派节点中拥有 “最完整” 的日志。

## 从当前任期提交条目

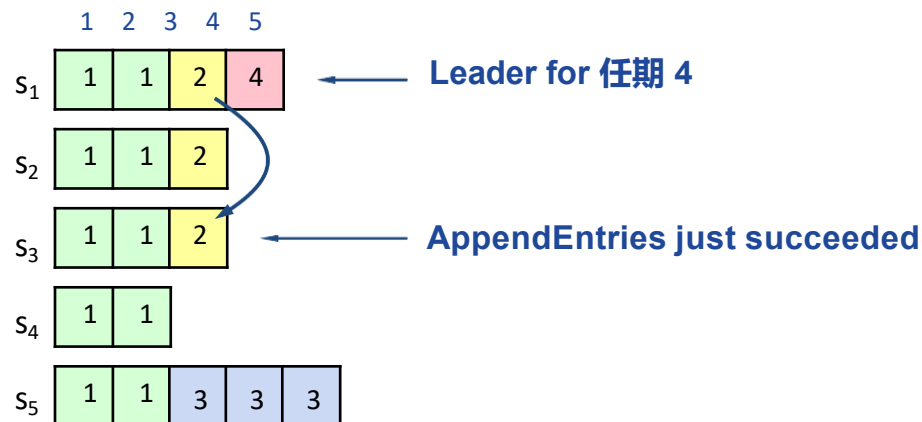


**案例 #1：领导者判定当前任期中的日志条目已提交**

- **安全性 (Safe) :**

在 **第 3 任期 (任期 3)** 当选的领导者 **必须包含日志条目 4**。

## 从更早的任期提交条目

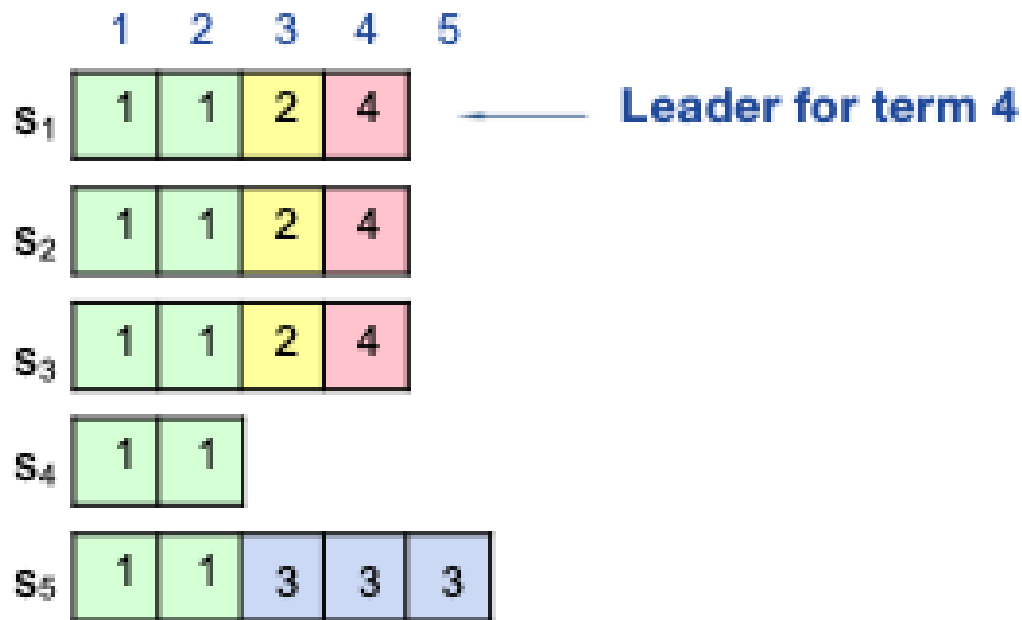


### 案例 #2：领导者试图完成提交较早任期中的日志条目

- 日志条目 3 并未被安全地提交 (not safely committed) :

- 节点 **s5** 仍然有可能在 **第 5 任期 (任期 5)** 被选举为领导者 (为什么?)
- 一旦 **s5** 当选为领导者, 它将会覆盖 (overwrite) 节点 **s1**、**s2** 和 **s3** 上的日志条目 3

## 新Commitment规则



- **领导者判定日志条目已提交 (committed) 的条件:**

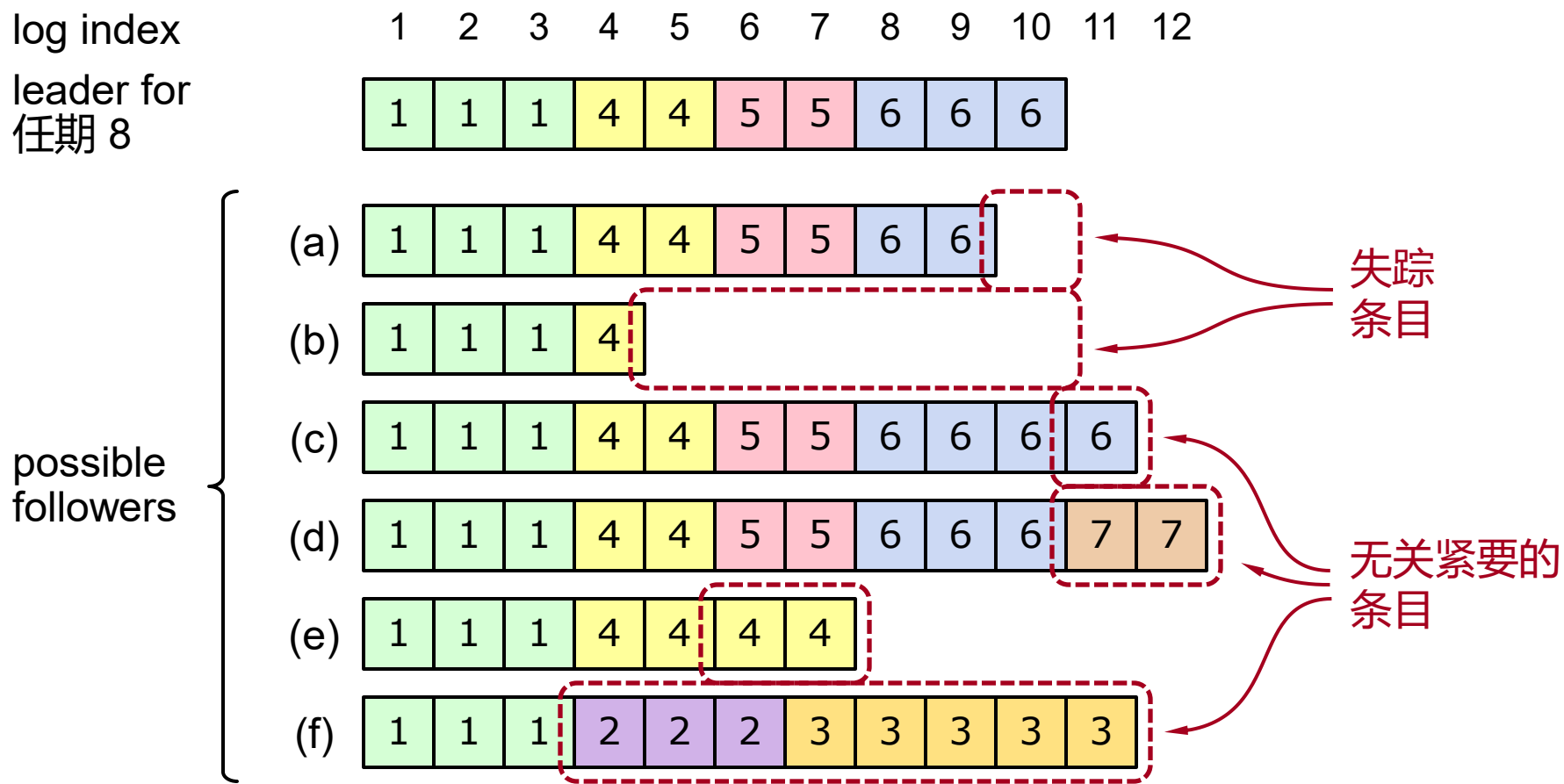
1. 该日志条目已被多数节点 (majority) 存储;
2. 同时, **领导者当前任期 (任期) 中的至少一个新日志条目**也已经被多数节点存储。

**示例:**

一旦日志条目 **e4** 被提交 (committed), 节点 **s5** 就不可能在 **第 5 任期 (任期 5)** 当选为领导者, 并且日志条目 **e3** 和 **e4** 都是**安全的 (safe)**。

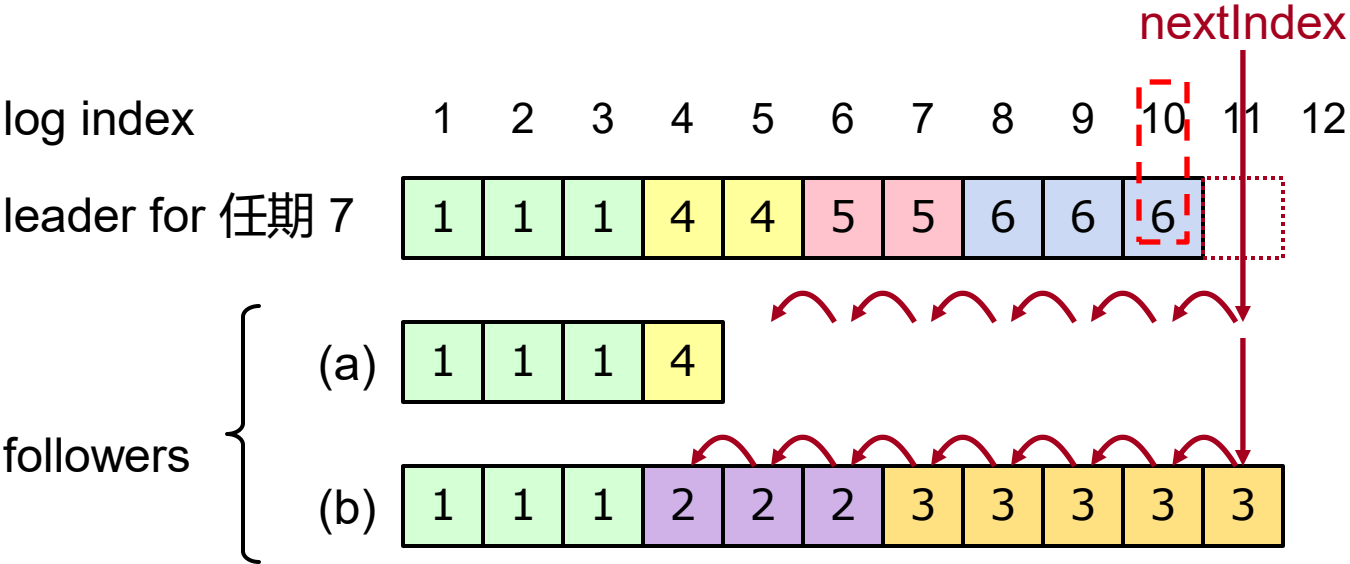
# 日志不一致

## 领导者的改变会导致日志不一致:



# 修复Follower Logs

- 领导者为每个关注者保留 **nextIndex**:
  - 下一个日志条目要发送给该从者的索引
- 当 **AppendEntries** 一致性检查失败时,递减 **nextIndex** 并重试:



- 当 **follower** 覆盖不一致的条目时,它会删除所有后续条目:



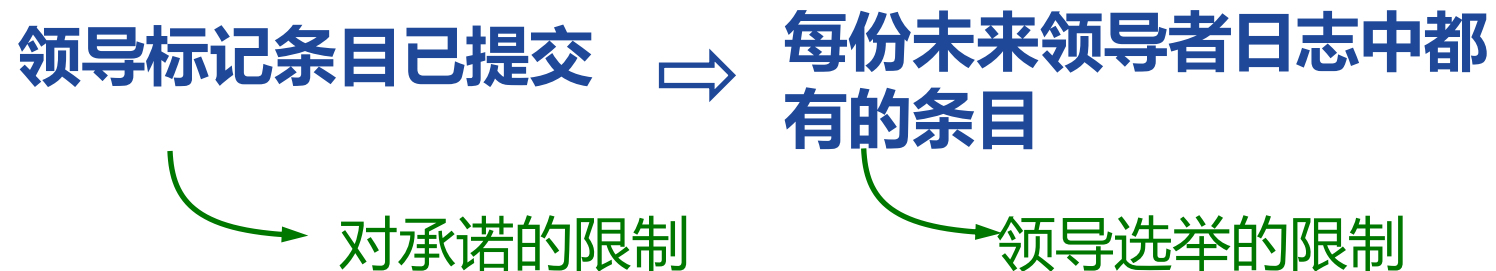


# 目录

## Contents

- 1 Raft共识算法
- 2 领导者选举
- 3 日志复制
- 3 安全性**

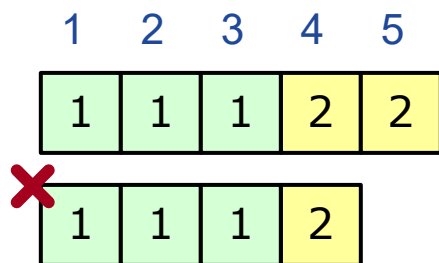
同一索引上任何两个已提交的条目都必须相同.



## 挑选最新的领导者

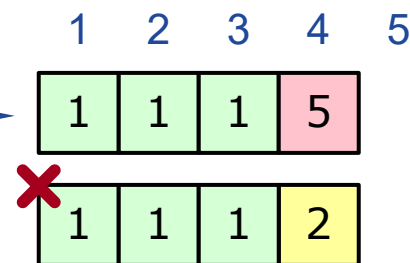
- 在选举期间,候选人必须在选举多数中拥有最新的日志:
  - 候选者在 RequestVote RPC 中包含日志信息 (日志的长度和最后的日志条目的期限)
  - 投票服务器如果日志更新,则拒绝投票:

Same last 任期 but  
different lengths:



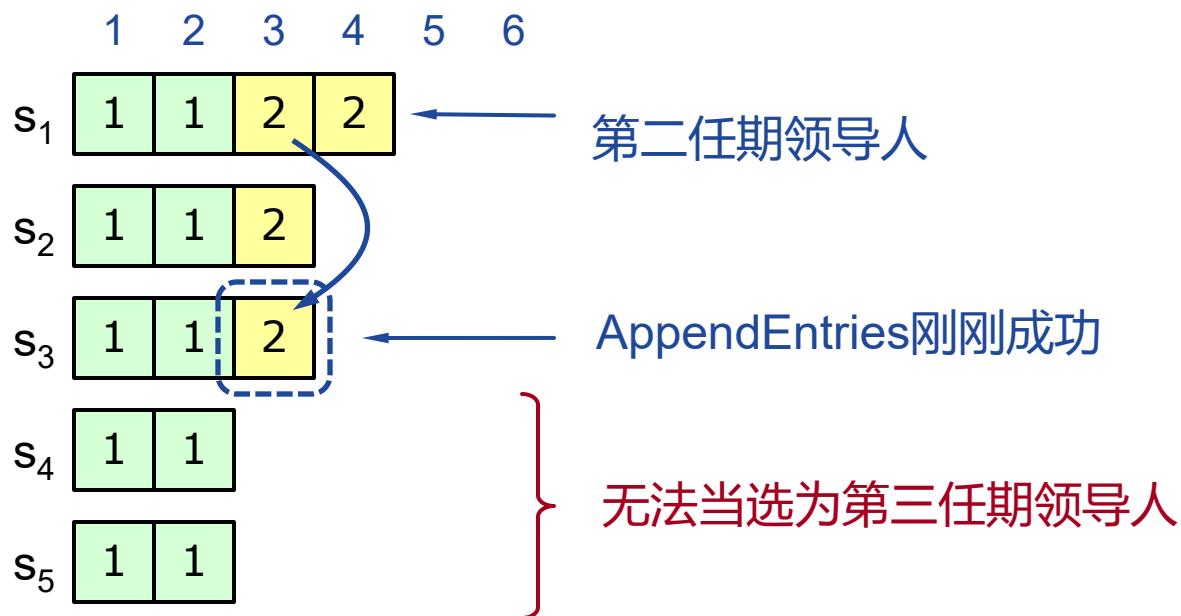
← more up-to-date →

Different last 任期:



## 从当前任期提交条目

- Case #1/2: 领导决定当前任期已承诺加入

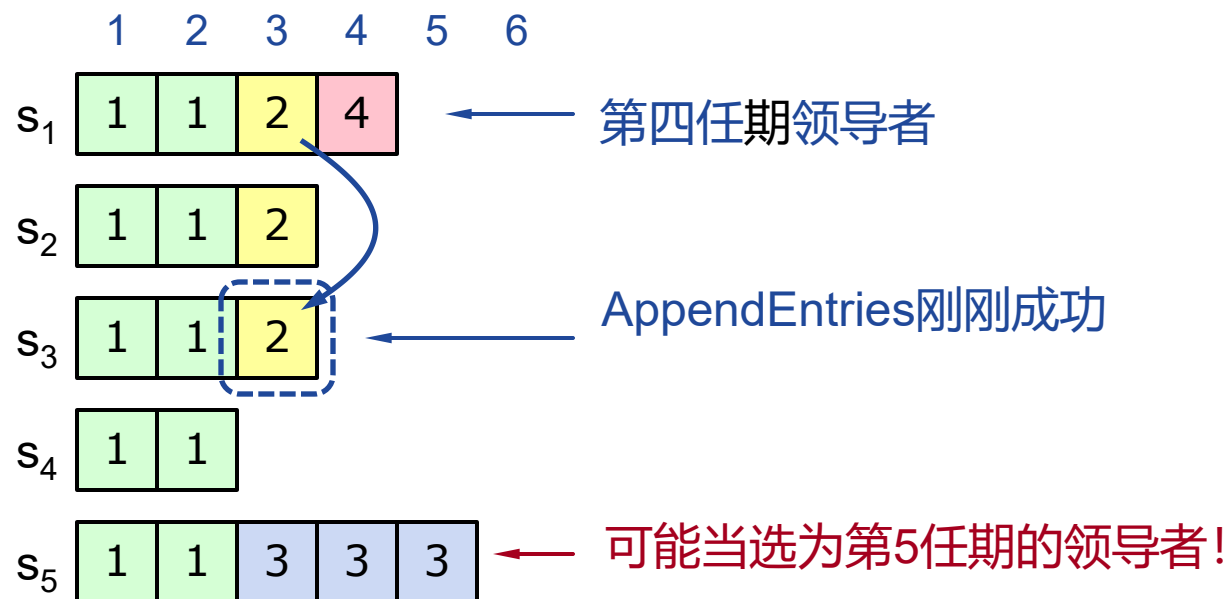


- 多数复制使条目 3 安全:

领导标记条目已提交  $\Rightarrow$  每份未来领导者日志中都有的条目

## 从前期提交条目

- Case #2/2: Leader 正试图完成前一任的条目

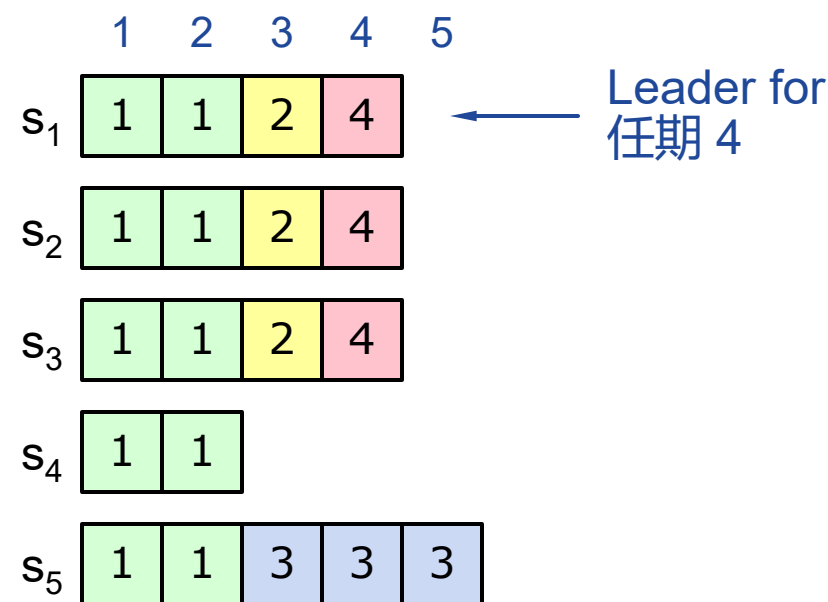


- 条目3未安全提交:

领导标记条目已提交  $\Rightarrow$  每份未来领导者日志中都有的条目

## 新的提交规则

- 新领袖在犯了本任的条目之前,不得标记已犯的旧条目.
- 一旦条目4提交:
  - $s_5$ 不能被选为第五任期的领导者
  - 条目3和4均安全



选举规则与提交规则的组合使Raft安全

1. 领袖选举
2. 正常运行
3. 安全

More at <https://raft.github.io/> :

- 论文中有更多细节（成员变更，对数压缩）
- 加入raft-dev邮件列表
- 可以看看GitHub上的25+实现版本